

Report 3: Hello, CUDA!

Le Nguyen Minh Quang – 2540093

September 30, 2026

1 Goal

In this labwork I convert a color image to a gray image in two ways: first on the CPU with a normal `for` loop, then on the GPU with `numba.cuda`. After that I compare the time of both versions and try different block sizes to see how they change the GPU time.

Like labwork 2, I ran the code on Google Colab because my laptop has no NVIDIA GPU. The GPU is a **Tesla T4**. My test image is a photo of a leaf, 549×364 pixels (199 836 pixels in total).

2 Method

- **Load the image:** I read the image with `plt.imread`. It has shape $(364, 549, 3)$, so I flatten it with `reshape(pixelCount, 3)`. Now each row is one pixel with its R, G and B values.
- **Gray value:** for each pixel I take the average of the three colors, $g = (R + G + B)/3$, and write g back to all three channels.
- **CPU version:** a simple `for` loop that goes through the pixels one by one.
- **GPU version:** each GPU thread handles one pixel. First I copy the image to the GPU with `cuda.to_device`, then I launch the kernel, and at the end I copy the result back with `copy_to_host`. The number of blocks is

$$\text{gridSize} = \lceil \text{pixelCount} / \text{blockSize} \rceil$$

- **Timing:** I used `time.time()`. I ran the kernel once before measuring, because the first call also compiles the kernel and is much slower. The GPU time includes copying the image to the GPU and back.

3 Code

```
# CPU
def grayscale_cpu(src):
    dst = np.empty_like(src)
    for i in range(pixelCount):
        g = (int(src[i, 0]) + int(src[i, 1]) + int(src[i, 2])) // 3
        dst[i, 0] = dst[i, 1] = dst[i, 2] = g
    return dst

# GPU
@cuda.jit
```

```
def grayscale_gpu(src, dst):
    tid = cuda.threadIdx.x + cuda.blockIdx.x * cuda.blockDim.x
    if tid < src.shape[0]:
        g = np.uint8((src[tid, 0] + src[tid, 1] + src[tid, 2]) / 3)
        dst[tid, 0] = dst[tid, 1] = dst[tid, 2] = g

devSrc = cuda.to_device(flatSrc)
devDst = cuda.device_array((pixelCount, 3), np.uint8)
grayscale_gpu(gridSize, blockSize)(devSrc, devDst)
gpuDst = devDst.copy_to_host()
```

I need the if in the kernel because the number of pixels is not always a multiple of the block size. The last block can have more threads than pixels, and those extra threads must do nothing.

4 Results

4.1 CPU vs GPU (block size 64)

	Time (ms)
CPU	256.19
GPU	1.85

So the GPU is about **138**× faster. To check the result, I compared the two gray images with `np.array_equal` and they are exactly the same. The pictures below also look the same.

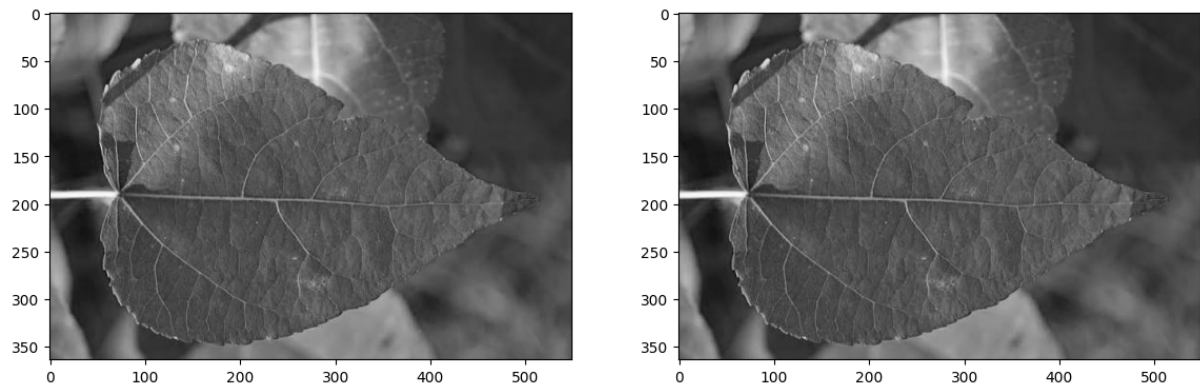


Figure 1: Gray image from the CPU (left) and from the GPU (right).

4.2 Block size vs time

I tried block sizes from 8 to 1024. One kernel call is very short, so for each block size I ran the kernel 50 times and took the average. Here I only measure the kernel, because the image is already on the GPU.

Block size	Grid size	Kernel time (ms)
8	24980	0.217
16	12490	0.110
32	6245	0.056
64	3123	0.053
128	1562	0.053
256	781	0.054
512	391	0.054
1024	196	0.057

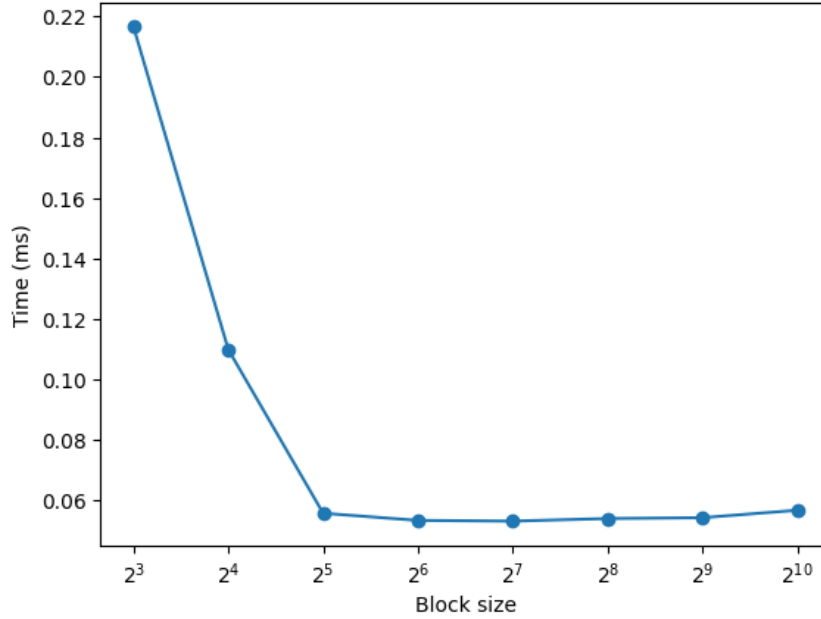


Figure 2: Kernel time vs block size.

5 Discussion

- The GPU is much faster than the CPU because it works on many pixels at the same time, while the CPU does them one by one.
- The GPU runs threads in groups of 32 (called a warp). With block size 8, only 8 of the 32 threads in each group do work, so it is about $4\times$ slower. With 16, only half of them work, so it is about $2\times$ slower.
- From 32 to 1024 every group is full, so the time is almost the same (about 0.055 ms).

6 Conclusion

The GPU version gives the same image as the CPU version and is about $138\times$ faster. The block size should be at least 32 (the warp size). After that, changing the block size does not change the time much.